

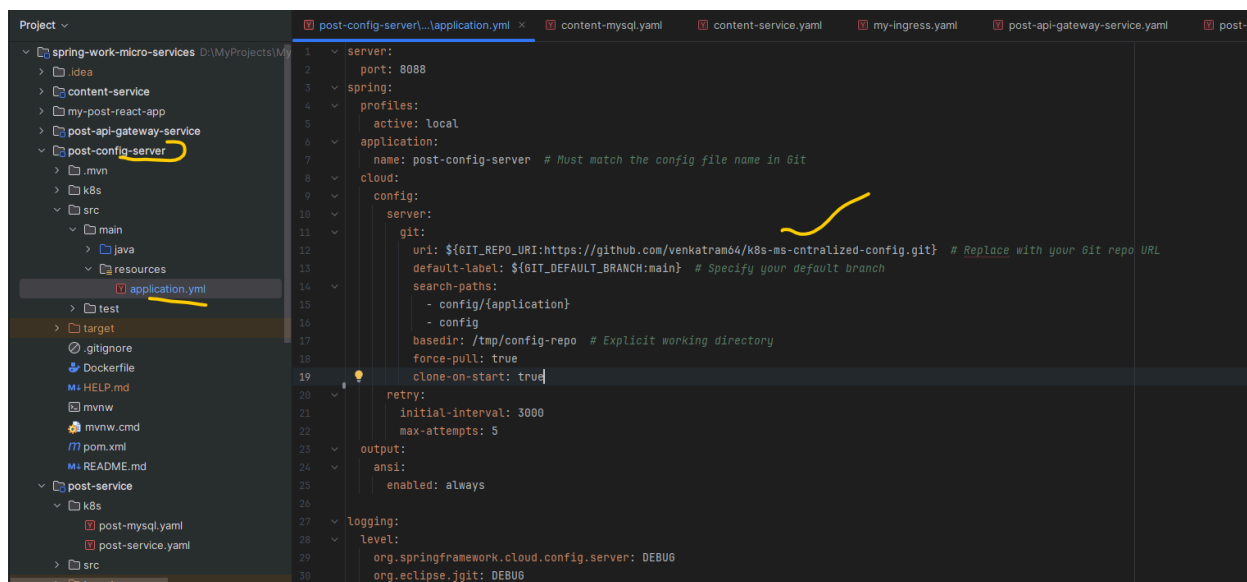
Complete end-to-end microservices implementation, Config is centralized in GitHub, and all the services are Kubernetes manifest file

Note: I installed Kubernetes on my local and deployed the microservices

Post and content microservices are registered with the Eureka service registry, and there is API gateway, also registered with the Eureka service registry. These service configurations hold the post-config server.

Config server:

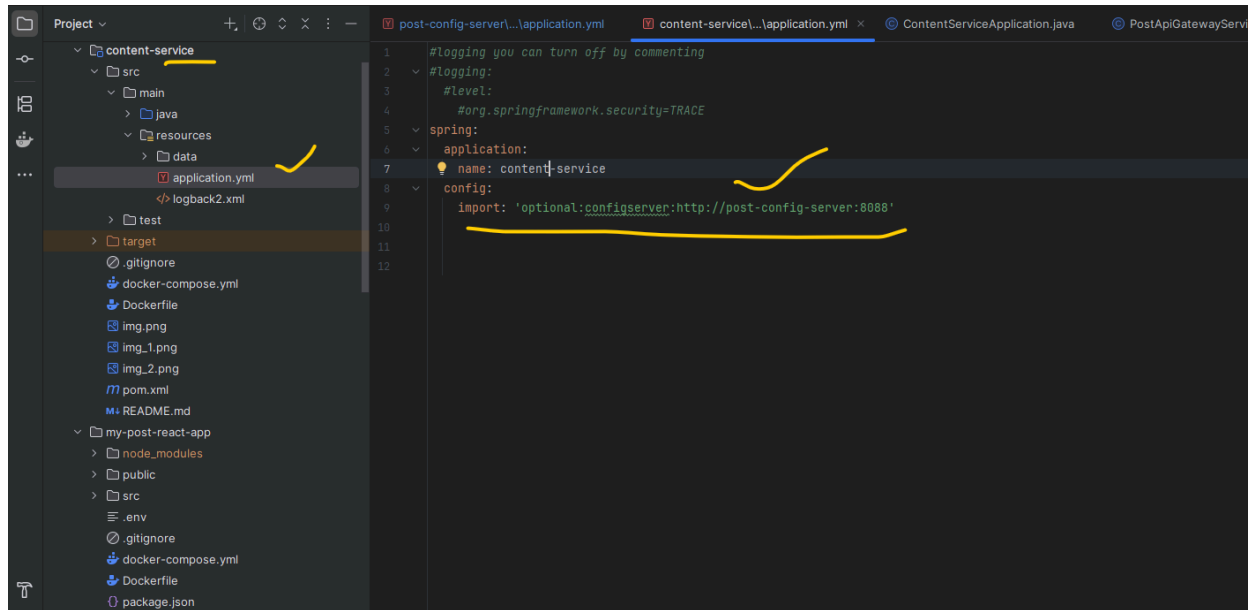
All the microservices' resource (config) information is moved into the config server's git location



The screenshot shows an IDE with a project structure on the left and a Kubernetes manifest file on the right. The project structure includes a folder named 'post-config-server' which contains 'application.yml'. The manifest file is 'application.yml' and contains the following configuration:

```
1 server:
2   port: 8088
3   spring:
4     profiles:
5       active: local
6     application:
7       name: post-config-server # Must match the config file name in Git
8   cloud:
9     config:
10      server:
11        git:
12          uri: ${GIT_REPO_URI:https://github.com/venkatram04/k8s-ms-centralized-config.git} # Replace with your Git repo URL
13          default-label: ${GIT_DEFAULT_BRANCH:main} # Specify your default branch
14          search-paths:
15            - config/{application}
16            - config
17          basedir: /tmp/config-repo # Explicit working directory
18          force-pull: true
19          clone-on-start: true
20      retry:
21        initial-interval: 3000
22        max-attempts: 5
23      output:
24        ansi:
25          enabled: always
26      logging:
27        level:
28          org.springframework.cloud.config.server: DEBUG
29          org.eclipse.jgit: DEBUG
```

Content-service.yml is used in content-service in the following way



Client Application that is UI or even when you are testing Post man, request is intercepted by the post-api-gateway service, see it's yml file, which is having routes information of two micro services.

For tracing zipkin is used, see it's configuration also

```
server:
  port: 8060

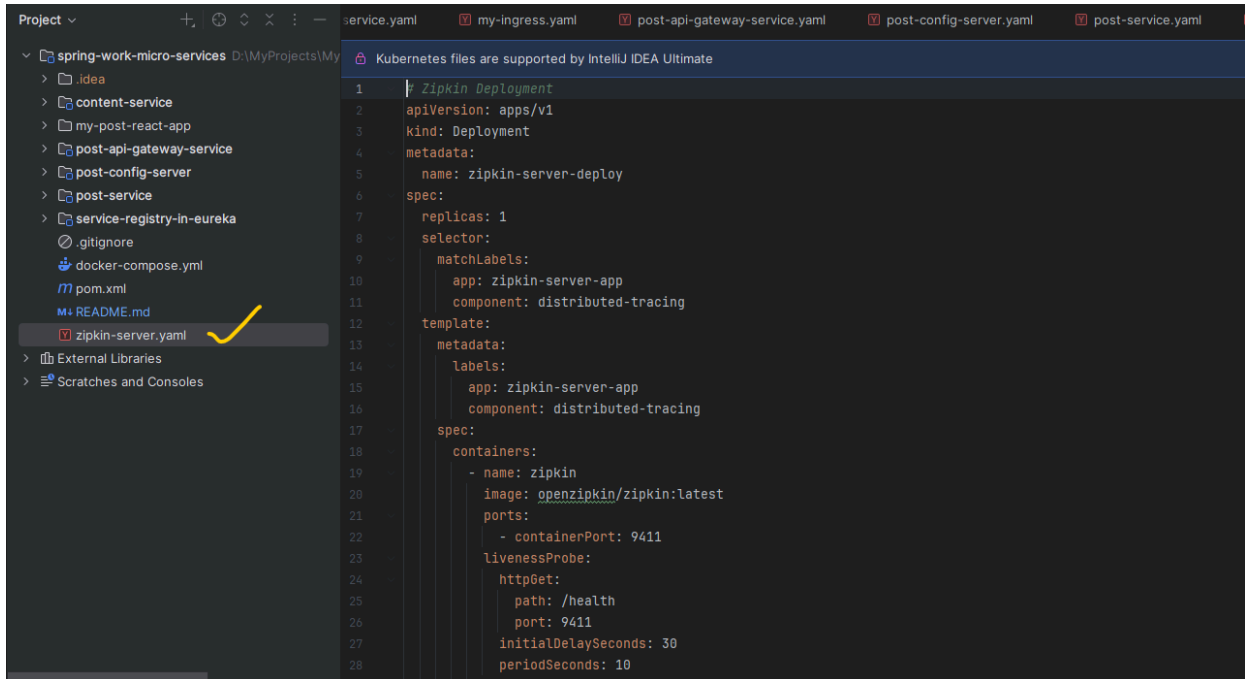
management: #actuator configuration
  endpoints:
    web:
      exposure:
        include: '*'
  endpoint:
    health:
      show-details: always
  tracing: #for zipkin logging
    sampling:
      probability: 1.0

eureka: #client is registering into Eureka server
  client:
    serviceUrl:
      defaultZone: http://localhost:8761/eureka/

spring:
  cloud:
    gateway:
      globalcors: #cors(cross origin resource sharing) configuration
      corsConfigurations:
        '[/*]':
          allowedOrigins: "*"
          allowedMethods: "GET,POST,PUT,DELETE,OPTIONS"
          allowedHeaders: "*"
      routes:
        - id: post-service #micro service application name
          uri: lb://post-service #micro service application name
          predicates:
            - Path=/api/posts/**
        - id: content-service #micro service application name
          uri: lb://content-service #micro service application name
          predicates:
            - Path=/api/auth/**, /api/content/**, /api/users/**

  output:
    ansi:
      enabled: always
```

For zipkin which is for tracing the log services across the micro service, I used deploy yaml to start this service which is configured into all the micro services.



The screenshot shows the IntelliJ IDEA interface with a project named 'spring-work-micro-services'. The file explorer on the left lists various files, including 'zipkin-server.yaml' which is highlighted with a yellow checkmark. The main editor displays the content of 'zipkin-server.yaml', a Kubernetes Deployment manifest. The manifest is titled 'Zipkin Deployment' and includes the following details:

- apiVersion:** apps/v1
- kind:** Deployment
- metadata:**
 - name:** zipkin-server-deploy
- spec:**
 - replicas:** 1
 - selector:**
 - matchLabels:**
 - app:** zipkin-server-app
 - component:** distributed-tracing
 - template:**
 - metadata:**
 - labels:**
 - app:** zipkin-server-app
 - component:** distributed-tracing
 - spec:**
 - containers:**
 - name:** zipkin
 - image:** openzipkin/zipkin:latest
 - ports:**
 - containerPort:** 9411
 - livenessProbe:**
 - httpGet:**
 - path:** /health
 - port:** 9411
 - initialDelaySeconds:** 30
 - periodSeconds:** 10

UI layer which is implemented in ReactJS.

See the below picture, interaction of microservices

And configuration is centralized using the git

<https://github.com/venkatram64/k8s-ms-centralized-config.git>

Clone the code and check out the "MB-1242-ms-k8s" branch, follow the README.md instructions

Started all the services(intelliJ idea), see pic

```
PS D:\MyProjects\MyWork\spring-work-micro-services> docker ps
CONTAINER ID   IMAGE                                COMMAND                                  CREATED        STATUS
PORTS         NAMES
ee58ba160750   kindest/node:v1.32.2              "/usr/local/bin/entr..." 2 weeks ago    Up 23
hours         0.0.0.0:443->443/tcp, 0.0.0.0:8080->80/tcp, 127.0.0.1:58514->6443/tcp
dev-cluster-control-plane
013539776d84   kindest/node:v1.32.2              "/usr/local/bin/entr..." 2 weeks ago    Up 23
hours         dev-cluster-worker2
561e08de0fd4   kindest/node:v1.32.2              "/usr/local/bin/entr..." 2 weeks ago    Up 23
hours         dev-cluster-worker
```

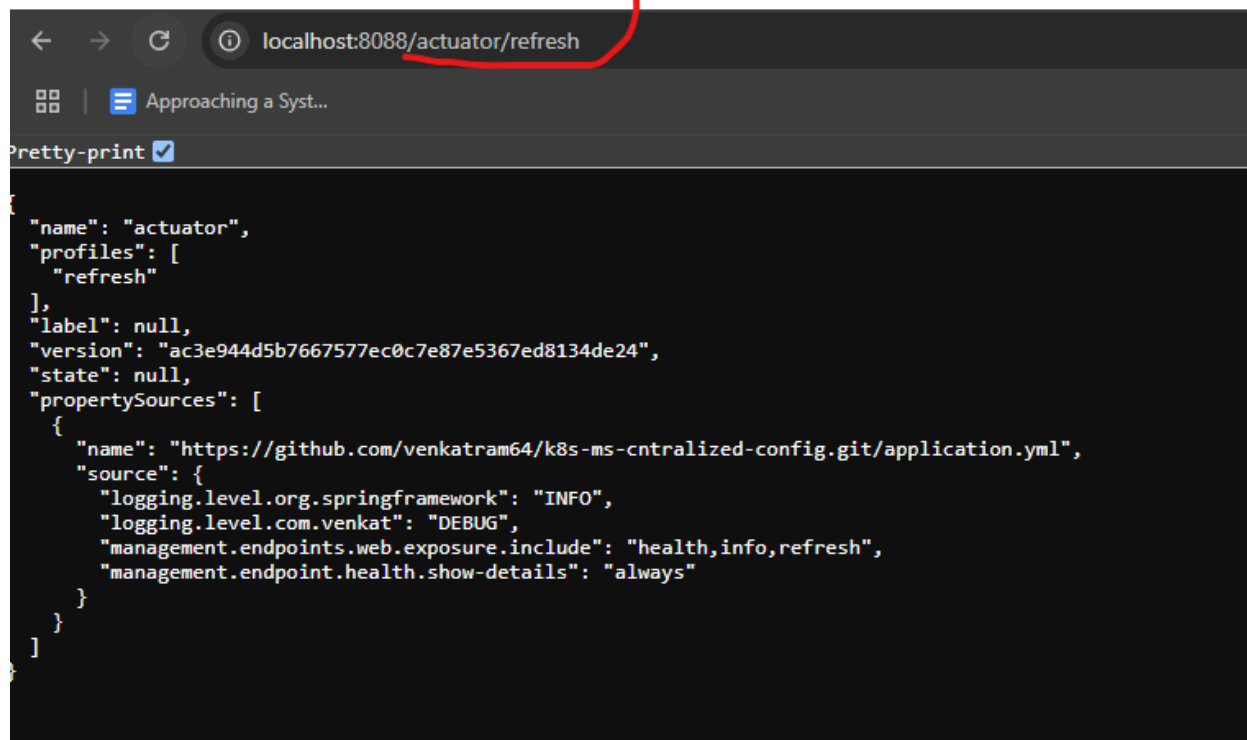
```
PS D:\MyProjects\MyWork\spring-work-micro-services> kubectl get nodes
NAME                                STATUS  ROLES    AGE  VERSION
dev-cluster-control-plane           Ready   control-plane  16d  v1.32.2
dev-cluster-worker                  Ready   <none>      16d  v1.32.2
dev-cluster-worker2                 Ready   <none>      16d  v1.32.2
PS D:\MyProjects\MyWork\spring-work-micro-services>
PS D:\MyProjects\MyWork\spring-work-micro-services>
```

See Starting the Order of services below

Step 1: Start zipkin server

```
PS D:\MyProjects\MyWork\spring-work-micro-services> kubectl apply -f .\zipkin-server.yaml
deployment.apps/zipkin-server-deploy created
service/zipkin-server created
PS D:\MyProjects\MyWork\spring-work-micro-services> |
```

To see the logs any use service name, see below as example



```
{
  "name": "actuator",
  "profiles": [
    "refresh"
  ],
  "label": null,
  "version": "ac3e944d5b7667577ec0c7e87e5367ed8134de24",
  "state": null,
  "propertySources": [
    {
      "name": "https://github.com/venkatram64/k8s-ms-cntralized-config.git/application.yml",
      "source": {
        "logging.level.org.springframework": "INFO",
        "logging.level.com.venkat": "DEBUG",
        "management.endpoints.web.exposure.include": "health,info,refresh",
        "management.endpoint.health.show-details": "always"
      }
    }
  ]
}
```

Then check the git updated property


```
localhost:8088/content-service/dev

Applying a System...

pretty-print

{
  "name": "content-service",
  "profiles": [
    "dev"
  ],
  "label": null,
  "version": "6c3e944d5b7667577ec0c7e87e5367ed8134de24",
  "state": null,
  "propertySources": [
    {
      "name": "https://github.com/venkatram64/k8s-ms-centralized-config.git/config/content-service-dev.yml",
      "source": {
        "spring.application.name": "content-service",
        "spring.config.activate.on-profile": "dev",
        "spring.zipkin.base-url": "http://zipkin-server:9411",
        "spring.zipkin.enabled": true,
        "spring.datasource.url": "jdbc:mysql://${MYSQL_HOST:content-mysql}:3306/content_db?useSSL=false&allowPublicKeyRetrieval=true",
        "spring.datasource.username": "${MYSQL_USER:root}",
        "spring.datasource.password": "${MYSQL_PASSWORD:root}",
        "spring.datasource.driver-class-name": "com.mysql.cj.jdbc.Driver",
        "spring.datasource.hikari.connection-timeout": 15000,
        "spring.datasource.hikari.minimum-idle": 2,
        "spring.datasource.hikari.maximum-pool-size": 5,
        "spring.datasource.hikari.idle-timeout": 10000,
        "spring.datasource.hikari.max-lifetime": 50000,
        "spring.jpa.hibernate.ddl-auto": "none",
        "spring.jpa.show-sql": true,
        "spring.jpa.properties.hibernate.format_sql": true,
        "spring.jpa.properties.hibernate.jdbc.time_zone": "UTC",
        "spring.jpa.database": "mysql",
        "spring.jpa.database-platform": "org.hibernate.dialect.MySQL8Dialect",
        "spring.output.ansi.enabled": "always",
        "eureka.client.register-with-eureka": true,
        "eureka.client.fetch-registry": true,
        "eureka.client.service-url.default-base": "http://service-registry-in-eureka:8761/eureka/",
        "eureka.client.healthcheck.enabled": true,
        "eureka.client.registry-fetch-interval-seconds": 5,
        "eureka.instance.prefer-ip-address": false,
        "eureka.instance.hostname": "${EUREKA_INSTANCE_HOSTNAME:content-service}",
        "eureka.instance.instance-id": "${eureka.instance.hostname}:${spring.application.name}:${server.port}",
        "eureka.instance.lease-renewal-interval-in-seconds": 10,
        "eureka.instance.lease-expiration-duration-in-seconds": 30,
        "jwtSecret": "6c3e9153b2ae0e5b20444bfa1a4b0db5fac666117d767dbd284b825ed85",
        "expirationInMs": 3600000,
        "logging.level.com.venkat": "DEBUG",
        "logging.level.org.springframework.cloud.config": "DEBUG"
      }
    },
    {
      "name": "https://github.com/venkatram64/k8s-ms-centralized-config.git/config/content-service.yml",
      "source": {
        "server.port": "${CONTENT_SERVICE_SERVER_PORT:8082}",
        "spring.profiles.active": "local",
        "spring.application.name": "content-service",
        "management.endpoints.web.exposure.include": "*",
        "management.endpoint.health.show-details": "always",
        "management.tracing.sampling.probability": 1
      }
    }
  ]
}
```

Step 3: Start the Eureka server

```
PS D:\MyProjects\MyWork\spring-work-micro-services\service-registry-in-eureka> docker build -t explorejava/service-registry-in-eureka .
[+] Building 17.7s (7/7) FINISHED
=> [internal] load build definition from Dockerfile
=> transferring dockerfile: 195B
=> [internal] load metadata for docker.io/library/openjdk:17-jdk-slim
=> [internal] load dockerignore
=> transferring context: 2B
=> [1/2] FROM docker.io/library/openjdk:17-jdk-slim@sha256:aaa3b3cb27e3e520b8f116863d0580c438ed55ecfa0bc126b41f68c3f62f9774
=> resolve docker.io/library/openjdk:17-jdk-slim@sha256:aaa3b3cb27e3e520b8f116863d0580c438ed55ecfa0bc126b41f68c3f62f9774
=> sha256:aaa3b3cb27e3e520b8f116863d0580c438ed55ecfa0bc126b41f68c3f62f9774 547B / 547B
=> sha256:779d3c3c621cc4dbab3d0c1ee4f2a922c199dfc8f4cb2799d2d493e0f22 293B / 953B
=> sha256:37cb4321d0423bc9726a3bfff658daf00478e4c4f2d3b8891f78510534256d 4.89kB / 4.89kB
=> sha256:44d3aa8d976675d49d85180b0ced9daf210fe4f4df4dbdb422b9cf384e591d0 1.58MB / 1.58MB
=> sha256:1fe172e4850f03b95d01a20174112bc119f4fec42a650eddbb8491aee32e3c3 31.38MB / 31.38MB
=> extracting sha256:1fe172e4850f03b95d01a20174112bc119f4fec42a650eddbb8491aee32e3c3 4.25s
=> extracting sha256:44d3aa8d976675d49d85180b0ced9daf210fe4f4df4dbdb422b9cf384e591d0 0.3s
=> sha256:6ce99df16e86bd02f6ad6a0e1334878528b5a4b5487850a76e0c08a7a27d56 187.90MB / 187.90MB
=> extracting sha256:6ce99df16e86bd02f6ad6a0e1334878528b5a4b5487850a76e0c08a7a27d56 5.6s
=> [internal] load build context
=> transferring context: 54.71MB
=> [2/2] COPY target/service-registry-in-eureka.jar app.jar
=> exporting to image
=> exporting layers
=> writing image sha256:f06f11b9f0e801fcacf60d0d58d5e6113ec01a4645597a95e55a9420a482
=> naming to docker.io/explorejava/service-registry-in-eureka 0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/jtboi4lkb0e3hmbpisklozmz

What's next:
  View a summary of image vulnerabilities and recommendations → docker scout quickview
PS D:\MyProjects\MyWork\spring-work-micro-services\service-registry-in-eureka> docker push explorejava/service-registry-in-eureka:latest
The push refers to repository [docker.io/explorejava/service-registry-in-eureka]
d3890b3da652: Pushed
6be99267647: Layer already exists
13a34b6ffff78: Layer already exists
9c1b6dd6c1e6: Layer already exists
latest digest: sha256:00c0dbd17610b708b9c2d10b10f3d081cc80b2b10b45da456a6e513b8723014 size: 116S
PS D:\MyProjects\MyWork\spring-work-micro-services\service-registry-in-eureka> kubectl apply -f .\k8s\
deployment.apps/service-registry-in-eureka-deploy created
service/service-registry-in-eureka created
PS D:\MyProjects\MyWork\spring-work-micro-services\service-registry-in-eureka> kubectl logs -f service/service-registry-in-eureka

[+] Spring Boot : (v3.2.1)
2025-07-06T06:47:38.485Z INFO 1 --- [service-registry-in-eureka] [main] c.v.ServiceRegistryInEurekaApplication : Starting ServiceRegistryInEurekaApplication using Java 17.0.2 with PID 1 (/app.jar)
```

Step 4: Start the Post Server

The below service takes more time, I used wait for database has to start, so please see the logs
Run the below service after starting the above service

```
PS D:\MyProjects\MyWork\spring-work-micro-services\post-service> docker build -t explorejava/post-service:latest .
[+] Building 4.8s (7/7) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 179B
=> [internal] load metadata for docker.io/library/openjdk:17-jdk-slim
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load build context
=> => transferring context: 74.40MB
=> CACHED [1/2] FROM docker.io/library/openjdk:17-jdk-slim@sha256:aaa3b3cb27e35e20b8f116863d0580c438ed55ecfa0bc126b41f68c3f62f9774
=> [2/2] COPY target/post-service.jar app.jar
=> exporting to image
=> => exporting layers
=> writing image sha256:baecc205420aab138f9199d0767f1cbfdd2647554a5e7ce151cbc41702e6dbd4
=> naming to docker.io/explorejava/post-service:latest
```

View build details: [docker-desktop://dashboard/build/desktop-linux/desktop-linux/fjls0trph4sdoaa3ea1u6nrov](#)

What's next:

View a summary of image vulnerabilities and recommendations → [docker scout quickview](#)

```
PS D:\MyProjects\MyWork\spring-work-micro-services\post-service> docker push explorejava/post-service:latest
```

The push refers to repository [docker.io/explorejava/post-service]

01b6bca691f6: Pushed

```
6be690267e47: Layer already exists
```

```
13a34b6fff78: Layer already exists
```

```
9c1b6dd6c1e6: Layer already exists
```

```
latest: digest: sha256:751d2309086086a7e26bfd6b425b4e484cff37bc2b6cedcb78165b1fb1aada43 size: 1165
```

```
PS D:\MyProjects\MyWork\spring-work-micro-services\post-service> kubectl apply -f .\k8s\
```

deployment.apps/posts-mysql-deploy unchanged

```
deployment.apps/posts-mysql-d
service/posts-mysql unchanged
```

```
service/posts-mysql unchanged
configmap/mysql-init-sql configured
```

```
configmap/mysql-init-sql configured
deployment.apps/post-service-deploy configured
```

```
deployment.apps/post-service-d
service/post-service unchanged
```

```
service/post-service unchanged
PS D:\MyProjects\MyWork\spring-work-micro-services\post-service>
```

```
PS D:\MyProjects\MyWork\spring-work-micro-services\post-service> kubectl logs -f service/post-service
Defaulted container "post-service-img" out of: post-service-img, wait-for-mysql (init)
Picked up JAVA_TOOL_OPTIONS: -Deureka.client.service-url.defaultZone=http://service-registry-in-eureka:8761/eureka/
```

[illegible]

```

2025-07-06T06:53:20.957Z INFO 1 --- [post-service] [main] [com.venkat.PostServiceApplication] Starting PostServiceApplication using Java 1
7.0.2 with PID 1 (/app.jar started by root in /)
2025-07-06T06:53:21.043Z DEBUG 1 --- [post-service] [main] [com.venkat.PostServiceApplication] Running with Spring Boot v3.2.1, Spring v6.1
2025-07-06T06:53:21.045Z INFO 1 --- [post-service] [main] [com.venkat.PostServiceApplication] The following 1 profile is active: "dev"
2025-07-06T06:53:21.259Z INFO 1 --- [post-service] [main] [o.s.c.c.c.ConfigServerConfigDataLoader] Fetching config from server at : http://post
-c-config-server:8888
2025-07-06T06:53:21.259Z INFO 1 --- [post-service] [main] [o.s.c.c.c.ConfigServerConfigDataLoader] Located environment: name=post-service, prof
iles=[default], label=null, version=f0988a054674fe954398db54045a081da8e46444, state=null
2025-07-06T06:53:21.259Z DEBUG 1 --- [post-service] [main] [o.s.c.c.c.ConfigServerConfigDataLoader] Environment post-service has 2 property sour
ces with 9 properties.
2025-07-06T06:53:21.259Z INFO 1 --- [post-service] [main] [o.s.c.c.c.ConfigServerConfigDataLoader] Fetching config from server at : http://post
-c-config-server:8888
2025-07-06T06:53:21.259Z INFO 1 --- [post-service] [main] [o.s.c.c.c.ConfigServerConfigDataLoader] Located environment: name=post-service, prof
iles=[dev], label=null, version=f0988a054674fe954398db54045a081da8e46444, state=null
2025-07-06T06:53:21.262Z DEBUG 1 --- [post-service] [main] [o.s.c.c.c.ConfigServerConfigDataLoader] Environment post-service has 3 property sour
ces with 41 properties.

```

Step 5: Start the content server

This service also takes time, please wait, after see the logs, start another one

```

PS D:\MyProjects\MyWork\spring-work-micro-services\content-service> docker build -t explorejava/content-service:latest .
[*] Building 6.1s (8/8) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 184B
=> [internal] load metadata for docker.io/library/openjdk:17-jdk-slim
[auth] library/openjdk:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load build context
=> => transferring context: 83.86MB
=> CACHED [1/2] FROM docker.io/library/openjdk:17-jdk-slim@sha256:aaa3b3cb27e3e520b8f116863d0580c438ed55ecfa0bc126b41f68c3f62f9774
=> [2/2] COPY target/content-service.jar app.jar
=> exporting to image
=> => exporting layers
=> => writing image sha256:11b10653e9bee6128b1ce5191b5ec7fd62a153183820a2d30dee3dd162f9ca5a
=> => naming to docker.io/explorejava/content-service:latest

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/v9t1l7vutjm750ufazzbbciyx

```

```

What's next:
View a summary of image vulnerabilities and recommendations → docker scout quickview
PS D:\MyProjects\MyWork\spring-work-micro-services\content-service> docker push explorejava/content-service:latest
The push refers to repository [docker.io/explorejava/content-service]
b93bf9e993ba: Pushed
6be690267e47: Layer already exists
13a34b6fff78: Layer already exists
9c1b6dd6c1e6: Layer already exists
latest: digest: sha256:7d5731500dc1b1bc98aced2f21f0a1035ef76c2d7a8ed36d6f23dda229c06483 size: 1165
PS D:\MyProjects\MyWork\spring-work-micro-services\content-service> kubectl apply -f .\k8s\
deployment.apps/content-mysql-deploy unchanged
configmap/mysql-init-sql configured
service/content-mysql unchanged
deployment.apps/content-service-deploy configured
service/content-service unchanged
PS D:\MyProjects\MyWork\spring-work-micro-services\content-service> |

```

```

PS D:\MyProjects\MyWork\spring-work-micro-services\content-service> kubectl apply -f .\k8s\
deployment.apps/content-mysql-deploy created
configmap/mysql-init-sql created
service/content-mysql created
deployment.apps/content-service-deploy created
service/content-service created
PS D:\MyProjects\MyWork\spring-work-micro-services\content-service> |

```

```

PS D:\MyProjects\MyWork\spring-work-micro-services\content-service> kubectl logs -f service/content-service
Defaulted container "content-service-1mg" out of: content-service-1mg, wait-for-mysql (init)
Picked up JAVA_TOOL_OPTIONS: -Deureka.client.service-url.defaultZone=http://service-registry-in-eureka:8761/eureka/

Spring Boot (v3.2.1)

2025-07-04T12:17:05.927Z INFO 1 --- [content-service] [main] [com.venkat.ContentServiceApplication] : Starting ContentServiceApplication using
Java 17.0.2 with PID 1 (/app.jar started by root in /)
2025-07-04T12:17:06.007Z DEBUG 1 --- [content-service] [main] [com.venkat.ContentServiceApplication] : Running with Spring Boot v3.2.1, Spring v
3.1.2
2025-07-04T12:17:06.008Z INFO 1 --- [content-service] [main] [com.venkat.ContentServiceApplication] : The following 1 profile is active: "dev"
2025-07-04T12:17:06.223Z INFO 1 --- [content-service] [main] [o.s.c.c.c.ConfigServerConfigDataLoader] : Fetching config from server at : http://p
ost-config-server:8080
2025-07-04T12:17:06.224Z INFO 1 --- [content-service] [main] [o.s.c.c.c.ConfigServerConfigDataLoader] : Located environment: name=content-service
, profiles=[default], label=null, version=ac3e944d5b7667577ec9c7e87e5367ed8134de24, state=null
2025-07-04T12:17:06.224Z DEBUG 1 --- [content-service] [main] [o.s.c.c.c.ConfigServerConfigDataLoader] : Environment content-service has 2 properti
y sources with 10 properties.
2025-07-04T12:17:06.307Z INFO 1 --- [content-service] [main] [o.s.c.c.c.ConfigServerConfigDataLoader] : Fetching config from server at : http://p
ost-config-server:8080
2025-07-04T12:17:06.307Z INFO 1 --- [content-service] [main] [o.s.c.c.c.ConfigServerConfigDataLoader] : Located environment: name=content-service
, profiles=[dev], label=null, version=ac3e944d5b7667577ec9c7e87e5367ed8134de24, state=null
2025-07-04T12:17:06.307Z DEBUG 1 --- [content-service] [main] [o.s.c.c.c.ConfigServerConfigDataLoader] : Environment content-service has 3 properti
y sources with 44 properties.

```

Step 6: Start the post api gate server

```

PS D:\MyProjects\MyWork\spring-work-micro-services\post-api-gateway-service> docker build -t explorejava/post-api-gateway-service:latest .
[+] Building 4.2s (7/7) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 193B
=> [internal] load metadata for docker.io/library/openjdk:17-jdk-slim
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load build context
=> => transferring context: 51.01MB
=> CACHED [1/2] FROM docker.io/library/openjdk:17-jdk-slim@sha256:aaa3b3cb27e3e520b8f116863d0588c438ed55ecfa0bc126b41f68c3f62f9774
=> [2/2] COPY target/post-api-gateway-service.jar app.jar
=> exporting to image
=> => exporting layers
=> => writing image sha256:be95d628a98aa30e667edf930dc0c552c138cc38e20e5b9d0d34a98fb5d47b5c
=> => naming to docker.io/explorejava/post-api-gateway-service:latest

```

View build details: [docker...desktop://dashboard/build/desktop-linux/desktop-linux/88goy2pm52ngzinc3e5ehqmy](https://dashboard.docker.com/build/desktop-linux/desktop-linux/88goy2pm52ngzinc3e5ehqmy)

What's next:

```

View a summary of image vulnerabilities and recommendations → docker scout quickview
PS D:\MyProjects\MyWork\spring-work-micro-services\post-api-gateway-service> docker push explorejava/post-api-gateway-service:latest
The push refers to repository [docker.io/explorejava/post-api-gateway-service]
d9b92dacfccf: Pushed
6be690267e47: Layer already exists
13a34b6ffff78: Layer already exists
9c1b6dd6c1e6: Layer already exists
latest: digest: sha256:2006ee6646266cf1de71f306912e178d16995ca7884fc55adb0652bd4dd7fcb8 size: 1165
PS D:\MyProjects\MyWork\spring-work-micro-services\post-api-gateway-service> kubectl apply -f .\k8s\
ingress.networking.k8s.io/my-post-ingress unchanged
deployment.apps/post-api-gateway-deploy configured
service/post-api-gateway-service unchanged
PS D:\MyProjects\MyWork\spring-work-micro-services\post-api-gateway-service> |

```

```

PS D:\MyProjects\MyWork\spring-work-micro-services\post-api-gateway-service> kubectl logs -f service/post-api-gateway-service
Picked up JAVA_TOOL_OPTIONS: -Deureka.client.service-url.defaultZone=http://service-registry-in-eureka:8761/eureka/

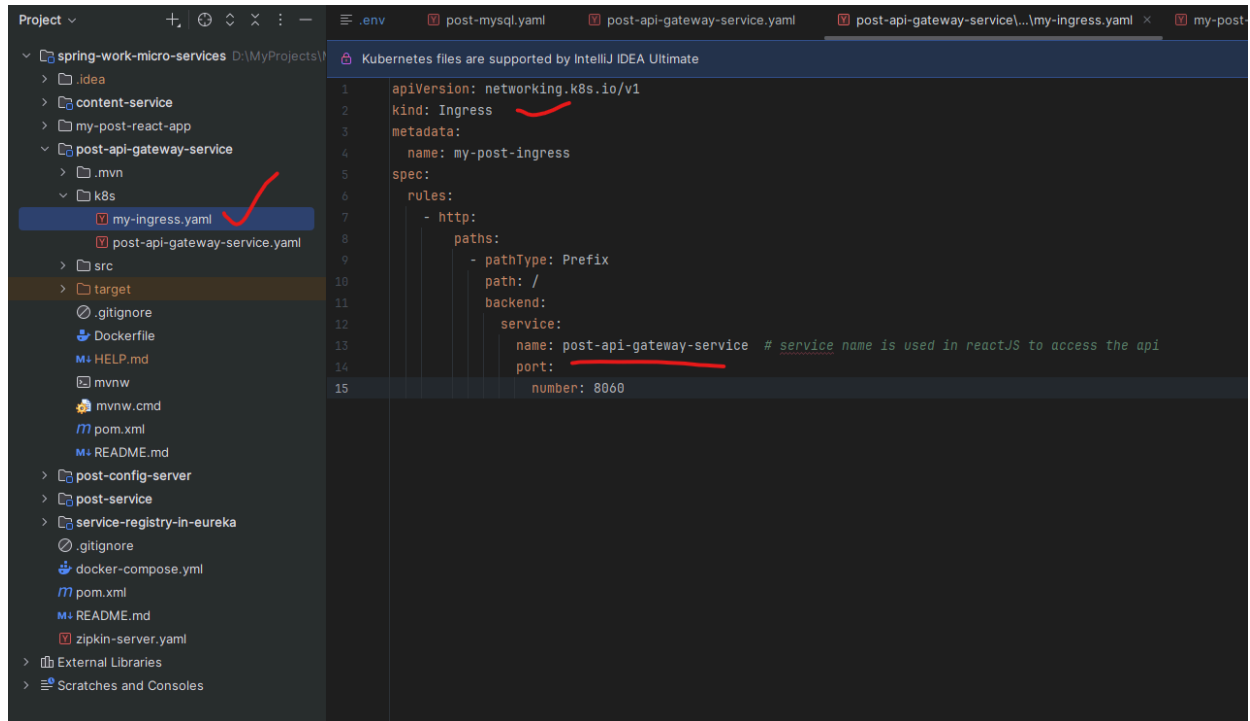
:: Spring Boot ::
(v3.2.0)

2025-07-08T12:20:25.286Z INFO 1 --- [post-api-gateway-service] [    main] [
plication v0.0.1-SNAPSHOT using Java 17.0.2 with PID 1 (/app.jar started by root in /)
2025-07-08T12:20:25.290Z DEBUG 1 --- [post-api-gateway-service] [    main] [
Spring v6.1.1
2025-07-08T12:20:25.291Z INFO 1 --- [post-api-gateway-service] [    main] [
s: "dev"
2025-07-08T12:20:25.349Z INFO 1 --- [post-api-gateway-service] [    main] [
http://post-config-server:8888
2025-07-08T12:20:25.349Z INFO 1 --- [post-api-gateway-service] [    main] [
ation, profiles=[default], label=null, version=ac3e904d5b7667877ecbc7e87e5357ed8134de24, state=null
2025-07-08T12:20:25.350Z INFO 1 --- [post-api-gateway-service] [    main] [
http://post-config-server:8888

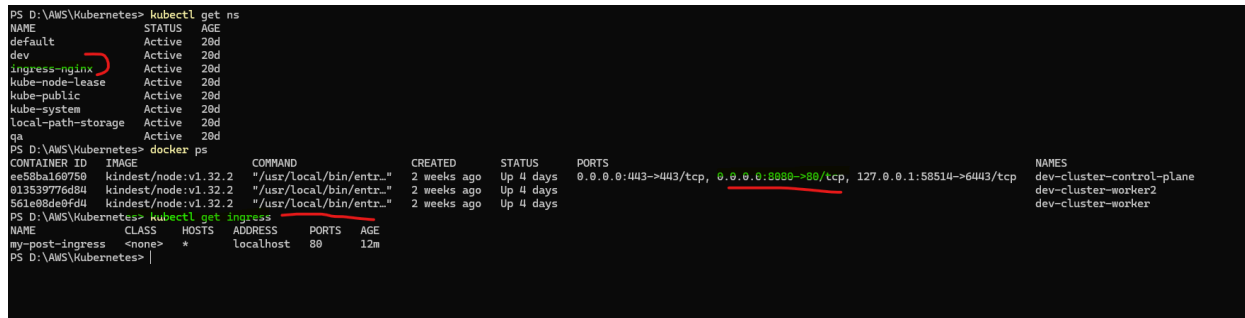
] c.v.p.PostApiGatewayServiceApplication : Starting PostApiGatewayServiceAp
] c.v.p.PostApiGatewayServiceApplication : Running with Spring Boot v3.2.0,
] c.v.p.PostApiGatewayServiceApplication : The following 1 profile is activ
] o.s.c.c.c.ConfigServerConfigDataLoader : Fetching config from server at :
] o.s.c.c.c.ConfigServerConfigDataLoader : Located environment: name=applic
] o.s.c.c.c.ConfigServerConfigDataLoader : Fetching config from server at :

```

Above server is exposed in ingress controller



I deployed ingress controller on my local



See all the pods/services are started

The screenshot shows the Spring Eureka dashboard. At the top, there's a navigation bar with the Spring Eureka logo and a link to 'HOME LAST 1000 SINCE STARTUP'. Below this, the 'System Status' section displays a table with system information:

Environment	test	Current time	2025-07-04T12:23:00 +0000
Data center	default	Uptime	00:31
		Lease expiration enabled	true
		Renews threshold	0
		Renews (last min)	14

Below the table, a red warning message states: 'THE SELF PRESERVATION MODE IS TURNED OFF. THIS MAY NOT PROTECT INSTANCE EXPIRY IN CASE OF NETWORK/OTHER PROBLEMS.'

The 'DS Replicas' section shows 'Instances currently registered with Eureka' in a table:

Application	AMIs	Availability Zones	Status
CONTENT-SERVICE	n/a (1)	(1)	UP (1) - content-service:content-service:8082
POST-API-GATEWAY-SERVICE	n/a (1)	(1)	UP (1) - post-api-gateway-service:post-api-gateway-service:8060
POST-SERVICE	n/a (1)	(1)	UP (1) - post-service:post-service:8081

The 'General Info' section shows a table with system metrics:

Name	Value
total-avail-memory	241mb

To run the my-post-react-app, your system should have the nodejs installation after that go to “my-post-react-app”

npm install

Above command will pull all the required modules for “my-post-react-app”
See the below screen shots for UI

The screenshot shows a terminal window with the following output:

```
venka@MSI MINGW64 /d/MyProjects/MyWork/spring-work-micro-services/my-post-react-app (MB-1239-security)
venka@MSI MINGW64 /d/MyProjects/MyWork/spring-work-micro-services/my-post-react-app (MB-1239-security)
$ npm install

up to date, audited 1652 packages in 12s

259 packages are looking for funding
  run `npm fund` for details

28 vulnerabilities (3 low, 9 moderate, 16 high)

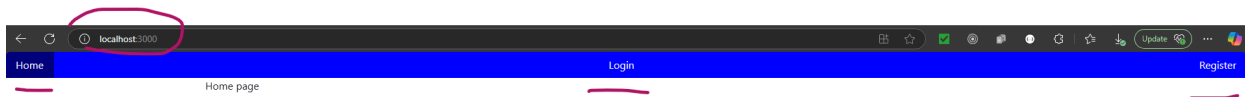
To address issues that do not require attention, run:
  npm audit fix

To address all issues (including breaking changes), run:
  npm audit fix --force
```

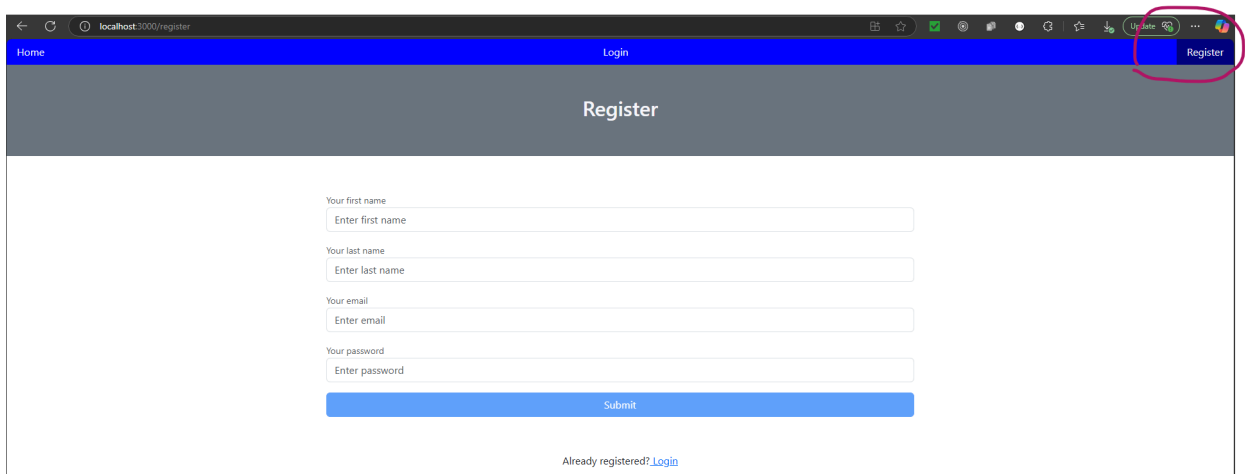
To run the app

npm run start

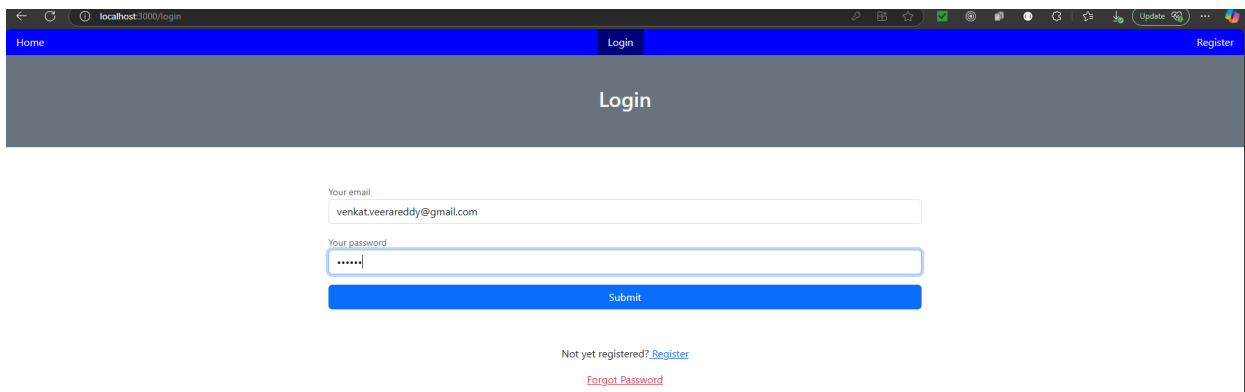
See the below pic in the browser, after you open link in browser <http://localhost:3000>



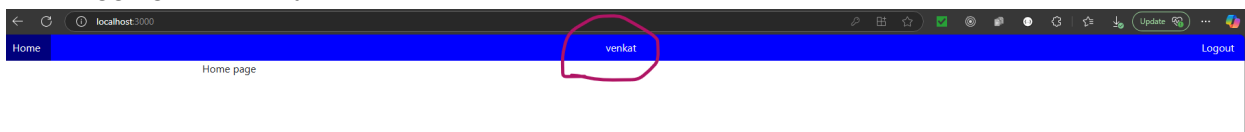
First register, see below page



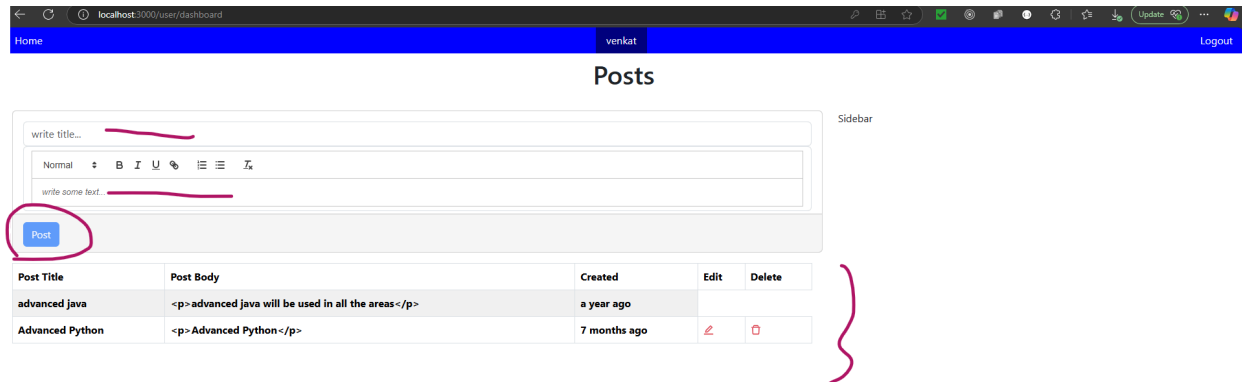
After that login into the system, in the following way



After logging into the system,



Click the logged in user name, above marked one
See the below posts, after creating the post screen



Please go through the each service [README.md](#) file and root level [README.md](#) file for build the docker images

For the entire code, the implementation will be available in the GitHub link.

<https://github.com/venkatram64/spring-work-micro-services/tree/MB-1242-ms-k8s>

And configuration is centralized using the git

<https://github.com/venkatram64/k8s-ms-centralized-config.git>

Images are stored in below link

<https://hub.docker.com/repositories/explorejava>

The screenshot shows the Docker Hub interface for the 'explorejava' namespace. The left sidebar contains navigation links: Repositories, Collaborations, Settings, Default privacy, Notifications, Billing, Usage, Pulls, and Storage. The main area is titled 'Repositories' and lists all repositories within the namespace. A search bar and a 'Create a repository' button are at the top right of the list.

Name	Last Pushed	Contains	Visibility	Scout
explorejava/post-api-gateway-service	about 1 hour ago	IMAGE	Public	Inactive
explorejava/my-post-react-app	about 2 hours ago	IMAGE	Public	Inactive
explorejava/content-service	1 day ago	IMAGE	Public	Inactive
explorejava/post-service	1 day ago	IMAGE	Public	Inactive
explorejava/service-registry-in-eureka	1 day ago	IMAGE	Public	Inactive
explorejava/post-config-service	1 day ago	IMAGE	Public	Inactive
explorejava/content-service	6 days ago	IMAGE	Public	Inactive
explorejava/post-config-server	7 days ago	IMAGE	Public	Inactive
explorejava/my-table	2 months ago	IMAGE	Public	Inactive
explorejava/k8s-web-to-nginx	about 2 years ago	IMAGE	Public	Inactive
explorejava/k8s-web-hello	about 2 years ago	IMAGE	Public	Inactive
explorejava/hello-world-rest-api	almost 5 years ago	IMAGE	Public	Inactive

To stop the deployed services in kubernetes

```
PS D:\MyProjects\MyWork\spring-work-micro-services> kubectl delete -f .\zipkin-server.yaml
deployment.apps "zipkin-server-deploy" deleted
service "zipkin-server" deleted
PS D:\MyProjects\MyWork\spring-work-micro-services> |
```

```
PS D:\MyProjects\MyWork\spring-work-micro-services\post-config-server> kubectl delete -f .\k8s\
deployment.apps "post-config-server-deploy" deleted
service "post-config-server" deleted
PS D:\MyProjects\MyWork\spring-work-micro-services\post-config-server> |
```

```
PS D:\MyProjects\MyWork\spring-work-micro-services\service-registry-in-eureka> kubectl delete -f .\k8s\
deployment.apps "service-registry-in-eureka-deploy" deleted
service "service-registry-in-eureka" deleted
PS D:\MyProjects\MyWork\spring-work-micro-services\service-registry-in-eureka> |
```

```
PS D:\MyProjects\MyWork\spring-work-micro-services\post-service> kubectl delete -f .\k8s\
deployment.apps "posts-mysql-deploy" deleted
service "posts-mysql" deleted
configmap "mysql-init-sql2" deleted
deployment.apps "post-service-deploy" deleted
service "post-service" deleted
PS D:\MyProjects\MyWork\spring-work-micro-services\post-service> |
```

```
PS D:\MyProjects\MyWork\spring-work-micro-services\content-service> kubectl delete -f .\k8s\
deployment.apps "content-mysql-deploy" deleted
configmap "mysql-init-sql" deleted
service "content-mysql" deleted
deployment.apps "content-service-deploy" deleted
service "content-service" deleted
PS D:\MyProjects\MyWork\spring-work-micro-services\content-service> |
```

```
PS D:\MyProjects\MyWork\spring-work-micro-services\post-api-gateway-service> ^C
PS D:\MyProjects\MyWork\spring-work-micro-services\post-api-gateway-service> kubectl delete -f .\k8s\
deployment.apps "post-api-gateway-deploy" deleted
service "post-api-gateway-service" deleted
PS D:\MyProjects\MyWork\spring-work-micro-services\post-api-gateway-service> |
```

```
PS D:\MyProjects\MyWork\spring-work-micro-services\my-post-react-app> kubectl delete -f .\k8s\
ingress.networking.k8s.io "my-post-ingress" deleted
deployment.apps "my-react-post-deploy" deleted
service "my-react-post-app-service" deleted
PS D:\MyProjects\MyWork\spring-work-micro-services\my-post-react-app> |
```

To delete all the images from my system and cache (please be careful, if you are using other images, please remove individually)

```
PS D:\MyProjects\MyWork\spring-work-micro-services> docker system prune -a
WARNING! This will remove:
- all stopped containers
- all networks not used by at least one container
- all images without at least one container associated to them
- all build cache

Are you sure you want to continue? [y/N] y
Deleted Images:
untagged: explorejava/post-config-service:latest
untagged: explorejava/post-config-service@sha256:9a2374d95aa133434e67a623be195c2906cf77ed2620222a7338a67f2a607042
deleted: sha256:e6b39f937aab20f394bbdbbe6c2f77f889082283338bcab90618648d4080c74d
deleted: sha256:aabcaa2f6739a8895916021d806a5fc6de3ba33a32c77bea0241d048f3ed3023
untagged: explorejava/post-api-gateway-service@sha256:b007ce061ff2d5dde0183a675b6c912f76d1236d880dfde9ca0024d63c85338f
deleted: sha256:0e4c51ffb1b122640cdc32390beba409c3679d912433c6631a4c1cb58108019c
untagged: explorejava/content-service:latest
untagged: explorejava/content-service@sha256:02cf4d4d321af65e50b03ca8a744c16c653f34971d6a1594539d21ee8e7d0ba2
deleted: sha256:e2032d7fe29f44bec2744f3b5191cb2a661a2167c224732ab04d4e3a0e54ea22
untagged: explorejava/my-post-react-app:latest
untagged: explorejava/my-post-react-app@sha256:47f74a6825319a7092ef743df374239d649deb4f522d02dfa7cddbdbb6241e43
deleted: sha256:8a4e2ad661ee68c06faa1752e01ad55d10b0c80b4b7e1b3b64ce85f6ddf22e49
untagged: explorejava/service-registry-in-eureka:latest
untagged: explorejava/service-registry-in-eureka@sha256:9931b82af39f62c7795ef336a544e9ab7ec07ade3da98d40d35fa7bc45ea9a8
deleted: sha256:09785b43b39d576d9a680a09334db74618295f744a7150bfe2810a823235db4a
untagged: explorejava/post-api-gateway-service@sha256:6bcd05db658582fca86aa02a5bf79e4819f5fe7cecb523bdcacaaab9dbb5d85be
deleted: sha256:6db6ddce4c7dad93b6cbb8e6d5f65eb47c9ffefce343d1d273d961fd2d4770e
untagged: explorejava/post-service:latest
untagged: explorejava/post-service@sha256:12d733f41f55644599f4b70cc4c9d736e756c57375abc235c1e6d850196feb3
deleted: sha256:e31673596cc14ff978434a609dc0a15e22af9d4082d38c4c9a5ad96c430f5430
untagged: explorejava/post-api-gateway-service@sha256:cddbbce47eac9e800642409df52ddeb846510fb766d21d1f92f02a6df09217c0
deleted: sha256:705d3e7603c31aefae8a9e4c9a392926095a2864fd2d2f3bca3742b1ebfc47c6
untagged: explorejava/post-api-gateway-service:latest
untagged: explorejava/post-api-gateway-service@sha256:349fd6d7c9d0d970ae887cd8096f7441fbac86c74a41989083eaff69109de913
deleted: sha256:1f3def5714a4783d8ece440c6cd973948615cc6781f290afd93a7d5d36080bf9

Deleted build cache objects:
irt3r4nx02s7ciz8vnky7uue
mwoy2b5vas7qyyj5cv365pvrd
xidhru544wshjkdz8e4ykgdpg
ld662tibvj3goxia3fa9dqzj6
j4rxn8lmivi6p7yo0qqwlb6c
q00ity6a3cimm4ij5o3ye587n
pr18r3bvgerivseqgk1i52qpb
u723484hexsbte19kb8cjw4p8
ry8mh9ah0nxwct5ygr445juvf
wpdir3somkis1pr0qip16uilo
uh72lstd0ef4io9272gctxg4ta
nvd5crklvc9ktp9b1i0v3c6l
lsh6qeaof9xpgum42iu0g9afa
2q554noiveavx3z0vi9gvc6dq
nu860kqhzn9m24diqinw8fbc
```